

Analyse des problèmes v4.1 → v5

Fichier : analyse_problemes_v4_1_v5.md **Dossier** : wiki racine
Branche : master (wiki) **Auteur** : hprzeta · **MAJ** : 2026-06-10

1. Problème — Mur de latence mpmath

Cause mathématique

`mpmath.siegelz(t, dps=35)` évalue $Z(t)$ via la formule de Riemann-Siegel avec $N = \lfloor \sqrt{t/2\pi} \rfloor$ termes à précision 35 dps (MPFR interne). Chaque appel déclenche $\sim 1\,000$ allocations/libérations heap (malloc/free MPFR) et un calcul en multi-précision (~ 160 bits).

Modèle de coût (§17 Formules_zeta.md) :

$$T_{\text{total}} = \frac{n \cdot n_{\text{iter}} \cdot t_{\text{appel}}}{W}$$

Avec $n = N(T_{\text{max}}) \approx 10\,142$ zéros, $n_{\text{iter}} \approx 8$, $W = 4$ workers, $t_{\text{appel}} = 21.13$ ms (mpmath) :

$$T_{\text{mpmath}} = \frac{10\,142 \times 8 \times 21.13}{4} \approx 430\,000 \text{ s} \approx 119 \text{ h}$$

Solution : Arb/FLINT double natif

`arb_fpwrap_cdouble_hardy_z` (libflint bundlée python-flint 0.8.0) reste en double IEEE 754 — aucune allocation heap, registres CPU uniquement :

$$t_{\text{arb}} = 0.77 \text{ ms} \Rightarrow T_{\text{arb}} \approx \frac{10\,142 \times 8 \times 0.77}{4} \approx 15\,600 \text{ s} \approx 4.3 \text{ h}$$

Avec $W=4$ et segmentation $1/\sqrt{t}$, la mesure réelle : **2.60 min** pour $T=10\,000$.

Méthode	t_{appel}	Speedup	T_{total} estimé (T=10k)
<code>mpmath.siegelz dps=35</code>	21.13 ms	1x	$\sim 7\text{h}$
<code>arb_fpwrap_cdouble_hardy_z</code>	0.77 ms	x27	$\sim 15 \text{ min}$
Mesuré (T=10k v2)	—	—	2.60 min

Benchmark détaillé : `src/benchmark/affinage_arb.py` (commit 074aef0), 200 points $t \in [100, 10\,000]$.

2. Problème — STEP fixe → zéros manquants

Cause mathématique

La densité des zéros de ζ sur la droite critique croît avec t :

$$\rho(t) = \frac{dN}{dt} \approx \frac{1}{2\pi} \ln \frac{t}{2\pi} \implies \delta(t) = \frac{2\pi}{\ln(t/2\pi)}$$

Un STEP fixe s rate les zéros si $s > \delta(t)/2$, c'est-à-dire si un bracket $[t, t + s]$ contient deux zéros (changement de signe double, indétectable).

Condition de non-manquant :

$$\text{STEP}(t) < \frac{\pi}{\ln(t/2\pi)}$$

Avec STEP=0.1 et $t = 100\,000$: $\pi / \ln(100\,000/2\pi) \approx 0.095$ — le STEP est juste à la limite. L'espacement minimum mesuré à $T=10\,000$ est $\delta_{\min} = 0.038$, donc STEP=0.1 est $2.6\times$ trop grand pour les paires de zéros proches.

Observation run T=100 000 (2026-06-10)

T	Zéros calculés	Attendus	Manquants
13 052	13 790	13 790	0
29 126	34 476	34 497	21
53 827	68 952	69 012	60
77 285	103 428	103 538	110
99 999	137 904	138 260	356

Aucun manquant jusqu'à $T=13\,052$ (là où STEP=0.1 est encore très supérieur à $\delta_{\min}$). Accumulation progressive au-delà, caractéristique d'un seuil de densité.

Solution : STEP adaptatif

Évolution de `step_pour_t(t)` dans `compute_zeros_v4_1.py` :

Version	Formule	T=10k	T=100k
v1 (commit 7467731)	paliers 0.1/0.05/0.02	6 manquants	30 manquants
v2 (commit 181fdd1)	cap 0.010 pour $t \geq 5\,000$	0 manquant ✓	$\times 11$ points de scan,
v3 (commit d2f62c1)	$\delta(t)/3$ continu, borné $[0.05; 0.5]$	—	EN COURS ✓

La v3 retenue :

$$\text{STEP}(t) = \max\left(0.05, \min\left(0.5, \frac{\delta(t)}{3}\right)\right)$$

Valeurs : ≈ 0.41 à $t = 1\,000$ · ≈ 0.33 à $t = 10\,000$ · ≈ 0.22 à $t = 100\,000$. Nombre de points de scan : $\approx 460\,000$ pour $T=100k$ (vs $5\,000\,000$ avec cap $0.010 - \div 11$).

Résultat $T=10\,000$ v2 (paliers $0.1/0.05/0.02$) → **0 manquant, Turing-Backlund COMPLET.**

3. Problème — GPU nVRTC GTX 960M

Cause

NVIDIA GTX 960M = Compute Capability 5.0 (sm_50). CUDA 12.x/NVRTC a déprécié sm_50 depuis la version 11.8 : `nVRTC: error: invalid value for --gpu-architecture (-arch).`

CuPy détecte la GPU, tente de compiler un kernel NVRTC et échoue silencieusement (le run continuait sur CPU numpy sans avertissement clair).

Solution

Dans `riemann_siegel_batch.py` (détection GPU, section 1) :

```
cc_major = props.get("major", 0)
if cc_major < 6:
    # sm_50/sm_52 → CUDA 12.x refuse, forcer CPU numpy
    print(f"GPU_{nom} (sm_{cc_major}...) : CC < 6.0 → mode CPU")
else:
    _GPU_DISPONIBLE = True
```

Résultat : message explicite au lieu d'une erreur runtime cryptique. Impact performance : `Z_batch` numpy reste rapide ($\times 7$ à $\times 15$ vs scalaire).

4. Problème — Déséquilibre de charge entre workers

Cause

La segmentation uniforme en t ($[14, 25k]$, $[25k, 50k]$, $[50k, 75k]$, $[75k, 100k]$) distribue des intervalles de même largeur. Or le nombre de zéros dans $[a, b]$ est :

$$N(b) - N(a) \approx \frac{b}{2\pi} \ln \frac{b}{2\pi e} - \frac{a}{2\pi} \ln \frac{a}{2\pi e}$$

Pour $T=100\,000$, Worker 0 traite $\sim 24\,900$ zéros, Worker 3 traite $\sim 38\,200$ zéros (ratio ~ 1.5). Le temps total est dominé par le worker le plus chargé.

Solution : segmentation $1/\sqrt{t}$

Couper l'axe $\sqrt{t}$ en N parts égales revient à égaliser $\int_{T_{j-1}}^{T_j} \frac{dt}{\sqrt{t}} = 2\sqrt{T_j} - 2\sqrt{T_{j-1}}$, ce qui approxime bien l'égalisation du nombre de zéros.

$$\text{Bornes : } T_j = \left(\sqrt{T_{\min}} + j \cdot \frac{\sqrt{T_{\max}} - \sqrt{T_{\min}}}{W} \right)^2$$

Implémenté dans `_partitionner_adaptatif()` — `compute_zeros_v4_1.py` (commit 7467731).

Segments pour $T=10\,000$ ($W=4$) :

Worker	Segment	Zéros estimés
0	[14, 775]	~760
1	[773, 2693]	~1 860
2	[2691, 5768]	~2 380
3	[5766, 10000]	~3 140

Le déséquilibre résiduel vient de la croissance logarithmique de $\rho(t)$; une segmentation par $N(T)$ serait parfaite mais plus coûteuse à calculer.

5. Tableau récapitulatif

Problème	Cause mathématique	Solution	Gain mesuré
Latence mpmath	MPFR heap, ~1000 malloc/appel	Arb double natif	x27 (0.77s)
STEP fixe	$\delta(t) = 2\pi / \ln(t/2\pi)$ décroît	STEP adaptatif 0.1/0.05/0.02	356 → 0 r
GPU nvrtc	sm_50 déprécié CUDA 12.x	Détection CC + fallback CPU	0 erreur
Load imbalance	Segments égaux, zéros inégaux	Segmentation $1/\sqrt{t}$	Workers :

6. Résultats mesurés

Run	STEP	T	Zéros	Manquants	Durée	Turing
v4.1 STEP fixe	0.1	10 000	10 137	6	2.58 min	INCOM
v4.1+Arb STEP v1	0.05 (t≥5k)	10 000	10 141	0	2.60 min	COMP
v4.1 STEP fixe	0.1	100 000	137 904	356	1h58	INCOM
v4.1+Arb STEP v2	0.1/0.05/0.02	100 000	138 039	68	105 min	INCOM
v4.1+Arb STEP v3	6/3	100 000	EN COURS	0 attendu	~3-4h	—

Gain v1 → v4.1+Arb : **x484** mesuré (21h → 2.60 min pour $T=10\,000$).

7. Questions ouvertes

1. **STEP = $\delta/3$ suffisant face aux paires très proches ?** Gap min mesuré : 0.019 à $t = 66\,678$, soit $\delta(66678)/3 \approx 0.22$ (ratio $\times 11.6$). La condition $\text{STEP} < \delta/2$ porte sur l'espacement *moyen* ; la queue GUE peut produire des gaps $\ll \delta$. **La validation Turing-Backlund du run v3 (EN COURS) tranchera.**
2. **Déséquilibre workers à grand t .** La segmentation $1/\sqrt{t}$ équilibre le nombre de zéros mais pas le temps d'affinage (Illinois croît en $O(\sqrt{t})$ par zéro). Worker 3 ([56 700, 100 000]) est $\sim 4\times$ plus lent. **→ v6 : segmenter par temps estimé $N(T) \cdot \sqrt{t_{\text{moyen}}}$.**
3. **Plancher de vitesse actuel : ~ 39 z/s cumulé (T=100k).** Pistes v6 : scan_arb.c (détection en C pur, -context switch Python/C) · W=8 · cible ~ 27 min.
4. **Odlyzko-Schönhage** : pertinent pour $T \gg 100\,000$ — réduit le coût asymptotique de $O(T^{1+\epsilon})$ à $O(T^{1/2+\epsilon})$ via FFT multi-évaluation.
5. **Précision** : zéro #20 LMFDB à $8.06\text{e-}10$ (cas limite stable depuis 3 runs). Investiguer si c'est un zéro proche de $t = 77.14$ ou une limitation de `illinois_refine`.

analyse_problemes_v4_1_v5.md · wiki racine · master · hprzeta · MAJ 2026-06-10 · 212 lignes